

JavaScript Pointer Like References

Objects • Arrays • Functions • Linked Lists • Real Time Programs

English + தமிழ் | JavaScript | Beginner to Interview Ready

CONCEPT

REAL TIME PROGRAMS

INTERVIEW FOCUS

Prepared as structured study notes with runnable examples and revision questions.

Start with the correct JavaScript fact

JavaScript does not provide C or C++ style explicit pointer syntax such as &a, *p, or manual pointer arithmetic. JavaScript-ல் objects, arrays, and functions reference values மூலம் access ஆகும்.

இதனால் pointer-like sharing and movement concepts புரிந்துகொள்ளலாம், ஆனால் raw memory address கிடைக்காது.

What this guide teaches

- Primitive value copy vs object reference copy
- Same object mutation and reference comparison
- Functions with shared object references
- Shallow copy and deep copy
- Linked List nodes using next references
- Real-time examples and interview-ready answers

One line memory trick

Expression	Simple meaning
Primitive copy	Value தனியாக copy ஆகும்
Object assignment	Same object-க்கு reference copy ஆகும்
current = current.next	Next object-க்கு reference move ஆகும்

1 Pointer meaning in general

In low-level languages, a pointer stores a memory address. JavaScript hides raw addresses for safety and automatic memory management. So JS code works with references controlled by the engine.

Feature	C or C++ pointer	JavaScript reference
Address operator	&a available	Not exposed
Dereference operator	*p available	Not exposed
Pointer arithmetic	Possible	Not available
Memory management	Often manual responsibilities	Garbage collected
Closest concept	Pointer to memory	Reference to object

2 Primitive copy

Primitive assignment copies the value. இரண்டு variables independent ஆக இருக்கும். One variable update செய்தால் other variable change ஆகாது.

```
let a = 10;
let b = a;
b = 50;
console.log(a); // 10
console.log(b); // 50
```

Mental diagram

Variable	Stored value	After b update
a	10	10
b	10 copy	50

3 Object reference copy

Object assignment creates another reference to the same object. இது independent clone இல்லை. Property mutation shared object-ஐ update செய்வதால் both variables வழியாக same result தெரியும்.

```
const user1 = { name: 'Arun', city: 'Chennai' };
const user2 = user1;
user2.city = 'Madurai';
console.log(user1.city); // Madurai
console.log(user2.city); // Madurai
```

Reference diagram

Variable	Reference target
user1	Student object A
user2	Student object A

Key idea: user1 and user2 are different variables, but their reference values identify the same underlying object.

4 Array reference behavior

```
const batchA = ['Asha', 'Bala'];
const batchB = batchA;
```

```
batchB.push('Charan');  
console.log(batchA); // ['Asha','Bala','Charan']  
console.log(batchA === batchB); // true
```

Array is also an object in JavaScript. So array assignment shares the same array reference unless a copy is explicitly created.

Create an independent top-level array

```
const original = ['Asha', 'Bala'];  
const copy = [...original];  
copy.push('Charan');  
console.log(original); // ['Asha','Bala']  
console.log(copy);      // ['Asha','Bala','Charan']
```

5 Reference equality

Same data but different objects

```
const x = { mark: 90 };  
const y = { mark: 90 };  
console.log(x === y); // false
```

Properties same இருந்தாலும் x and y இரண்டு separate object identities. Triple equals deep content compare செய்யாது.

Same object reference

```
const x = { mark: 90 };  
const y = x;  
console.log(x === y); // true
```

6 Functions and shared references

JavaScript is pass-by-value. Object argument case-ல் copied value itself is a reference. Function அந்த shared object-ன் property-ஐ mutate செய்யலாம்.

```
function addBonus(employee) {  
  employee.salary += 5000;  
}  
const staff = { name: 'Asha', salary: 30000 };  
addBonus(staff);  
console.log(staff.salary); // 35000
```

Mutation vs reassignment

```
function tryReplace(person) {  
  person = { name: 'New User' }; // local reassignment only  
}
```

```
const user = { name: 'Original' };
tryReplace(user);
console.log(user.name); // Original
```

Interview wording: "The function receives a copy of the reference value. Mutating the referenced object is visible outside; reassigning the parameter is not."

7 const does not freeze an object

```
const profile = { name: 'Asha' };
profile.name = 'Divya'; // allowed
// profile = { name: 'New' }; // TypeError: reassignment not allowed
```

const protects the variable binding from reassignment. Object properties remain mutable unless Object.freeze or another immutability strategy is used.

Freeze basics

```
const settings = Object.freeze({ theme: 'dark' });
settings.theme = 'light'; // ignored or error in strict mode
console.log(settings.theme); // dark
```

Object.freeze is shallow. Nested objects are not automatically frozen.

8 Shallow copy

```
const student1 = {
  name: 'Asha',
  address: { city: 'Chennai' }
};
const student2 = { ...student1 };
student2.name = 'Bala';
student2.address.city = 'Madurai';
console.log(student1.name); // Asha
console.log(student1.address.city); // Madurai
```

Spread creates a new outer object, but nested address reference is copied. அதனால் nested mutation original-லும் reflect ஆகிறது.

Visual map

Part	student1 vs student2
Outer object	Different references
name primitive	Independent copied value
address object	Shared nested reference

9 Deep copy

```
const original = {
  name: 'Asha',
  address: { city: 'Chennai' }
};
const deepCopy = structuredClone(original);
deepCopy.address.city = 'Madurai';
console.log(original.address.city); // Chennai
```

`structuredClone` creates independent nested data for supported cloneable values. Functions cannot be cloned with it. Custom class behavior and special objects need case-by-case handling.

10 Garbage collection and reachability

JavaScript automatically reclaims objects that are no longer reachable. Developer raw memory free செய்ய மாட்டார். But keeping unnecessary references in global variables, caches, timers, or event listeners can keep objects reachable and cause memory growth.

```
let session = { user: 'Asha', largeData: new Array(1000) };
session = null;
// If no other references exist, the old object becomes eligible for GC.
```

Important: “Eligible for garbage collection” does not mean memory is freed immediately. The engine decides when collection runs.

11 Pointer like Node concept

DSA-ல் node object contains data plus references to other nodes. next property raw pointer இல்லை; it stores another object reference or null.

```
const node1 = { value: 10, next: null };
const node2 = { value: 20, next: null };
const node3 = { value: 30, next: null };
node1.next = node2;
node2.next = node3;
```

Node chain

Node	value	next points to
node1	10	node2
node2	20	node3
node3	30	null

12 Traverse a Linked List

```
let current = node1;
while (current !== null) {
  console.log(current.value);
  current = current.next;
}
// Output: 10, 20, 30
// Time O(n), extra space O(1)
```

`current = current.next` means current reference now identifies the next node. இது pointer movement மாதிரி conceptual behavior, ஆனால் address arithmetic இல்லை.

Dry run

Iteration	current value	Next current
1	10	node2
2	20	node3
3	30	null

13 Build a reusable Linked List

```
class Node {
  constructor(value) {
    this.value = value;
    this.next = null;
  }
}
class LinkedList {
  constructor() {
    this.head = null;
  }
  append(value) {
    const node = new Node(value);
    if (!this.head) { this.head = node; return; }
    let current = this.head;
    while (current.next) current = current.next;
    current.next = node;
  }
  print() {
    let current = this.head;
    while (current) { console.log(current.value); current = current.next; }
  }
}
const list = new LinkedList();
list.append(10); list.append(20); list.append(30);
list.print();
```

append in this simple version is $O(n)$ because it walks to the tail. Tail reference maintain செய்தால் append-ஐ $O(1)$ ஆக்கலாம்.

14 Real Time Programs

Program 1 Shopping cart shared state bug

```
const cart = { items: ['Book'], total: 500 };
const checkoutPreview = cart;
checkoutPreview.items.push('Pen');
console.log(cart.items); // ['Book','Pen'] - original changed
```

Problem: preview edit original cart-ஐ change செய்கிறது because both share one object. Safe top-level and nested copy below.

```
const checkoutSafe = { ...cart, items: [...cart.items] };
checkoutSafe.items.push('Bag');
console.log(cart.items); // ['Book','Pen']
console.log(checkoutSafe.items); // ['Book','Pen','Bag']
```

Program 2 React style immutable update

```
const state = {
  students: [{ id: 1, name: 'Asha', mark: 70 }]
};
const nextState = {
  ...state,
  students: state.students.map(s =>
    s.id === 1 ? { ...s, mark: 90 } : s
  )
};
console.log(state.students[0].mark); // 70
console.log(nextState.students[0].mark); // 90
```

React state update-ல் existing state mutate செய்யாமல் new references create பண்ணுவது change detection and predictable updates-க்கு important.

Program 3 Employee manager relationship

```
const manager = { id: 1, name: 'Meena' };
const employee = { id: 2, name: 'Arun', manager };
manager.name = 'Meena S';
console.log(employee.manager.name); // Meena S
```

employee.manager and manager identify the same manager object. Backend normalized data, caches, and relational UI models-ல் இந்த sharing useful, ஆனால் accidental mutation கவனிக்க வேண்டும்.

Program 4 Browser history nodes

```
const home = { page: 'Home', next: null, prev: null };
const courses = { page: 'Courses', next: null, prev: home };
home.next = courses;
const detail = { page: 'Course Detail', next: null, prev: courses };
courses.next = detail;
console.log(detail.prev.prev.page); // Home
```

This is a doubly linked relationship: next forward navigation, prev backward navigation.

Program 5 Detect a cycle

```
function hasCycle(head) {  
  let slow = head, fast = head;  
  while (fast && fast.next) {  
    slow = slow.next;  
    fast = fast.next.next;  
    if (slow === fast) return true;  
  }  
  return false;  
}  
// Floyd algorithm: time O(n), extra space O(1)
```

15 Common Mistakes

Mistake	Correct understanding
"JS has pointers like C"	No explicit/raw pointers; use object references
"Objects are passed by reference"	JS passes values; object value is a reference
"Spread is deep copy"	Spread is shallow
"const object cannot change"	Binding cannot reassign; properties can mutate
"Equal properties mean objects are ==="	Equality checks object identity
"null means deleted memory"	It is a value; GC depends on reachability

16 Interview Questions and Answers

1 Does JavaScript have pointers?

Not explicit pointers. It uses references to objects while hiding raw memory addresses.

2 What is a reference value?

A value that lets JavaScript identify and access an object managed by the engine.

3 Why does changing user2 affect user1?

Both variables contain reference values for the same object, so the mutation targets one shared object.

4 Is JavaScript pass-by-reference?

No. It is pass-by-value; for objects, that value is a reference.

5 What does const protect?

It prevents reassignment of the binding, not mutation of object properties.

6 What is shallow copy?

A new outer container with nested references potentially shared.

7 How do you deep clone in modern JavaScript?

Use structuredClone for supported data types; choose domain-specific cloning for unsupported values.

8 Why is {} === {} false?

Each object literal creates a separate object identity.

9 How does Linked List use references?

Each node stores next, and optionally prev, as references to other node objects.

10 What is a memory leak risk in JS?

Unneeded objects remain reachable through listeners, timers, caches, closures, or globals.

11 What is the append complexity in the shown list?

$O(n)$ without a tail reference; $O(1)$ is possible with a maintained tail.

12 Can you perform pointer arithmetic in JS?

No. JavaScript does not expose raw addresses or pointer arithmetic.

17 Practice Programs

1. Show primitive copy using two mark variables.
2. Show same-object mutation using two student variables.
3. Prove two separate equal-looking objects are not strictly equal.
4. Create an independent array copy using spread.
5. Demonstrate shallow-copy nested reference sharing.
6. Deep clone supported data using structuredClone.
7. Write a function that mutates an employee salary.
8. Write a function that reassigns its parameter and explain why caller does not change.
9. Create three linked nodes and traverse them.
10. Add prepend, find, and delete methods to LinkedList.
11. Maintain tail so append becomes $O(1)$.
12. Create a doubly linked browser-history example.

Final cheat sheet

Code	Meaning
let b = a primitive	Value copy
const b = a object	Same object reference
a === b objects	Same identity check
{ ...a }	New outer object, shallow copy
structuredClone a	Independent clone for supported data
node.next = anotherNode	Reference link
current = current.next	Move to next referenced node

Best interview summary: "JavaScript hides memory addresses. Primitive assignment copies the primitive value. Object assignment copies a reference value, so multiple variables can access the same object. Linked structures are built by storing object references in properties such as next and prev."